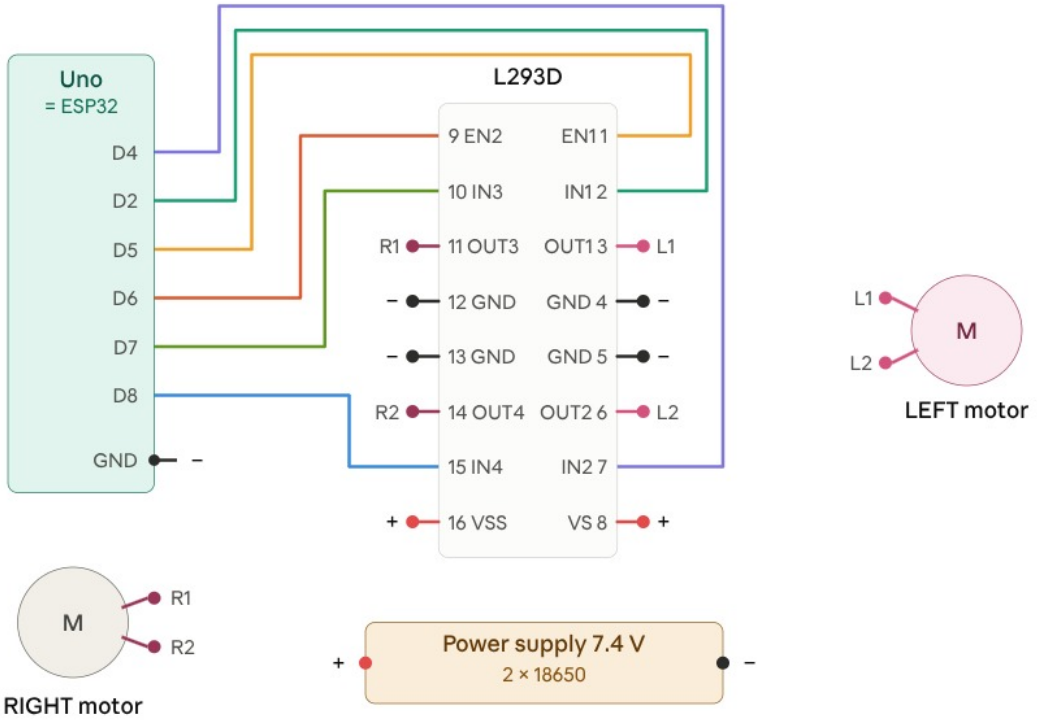
UGV
PROGRESS LOG

-BHAVAJNA,LASYA

Initially, we tested the motors by connecting them in series and powering the circuit using a battery. Both motors started rotating successfully, confirming that the motors were functioning properly.

After this, we built the given circuit:



and uploaded the code to operate the motors without Bluetooth. During testing, one wheel started rotating while the other remained stationary (we did not observe this). As the motors were spinning and as the code had only a 2seconds gap to switch between forward and backwards and we thought that was not enough to recognise those commands. So we just assumed it would work fine with the bluetooth code and proceeded to implement Bluetooth control. During further testing, we observed that the circuit operated without receiving any commands and that one

wheel was not rotating at all. We suspected that the issue might be related to faulty breadboard connections and decided to investigate further.

To check whether the breadboard was causing the issue, we tested a seven-segment display to verify that the breadboard rows were functioning correctly. We placed the IC among the working rows. However, the motor circuit still did not work as expected. We then exchanged the positions of the two motors. After swapping them, the wheel that had previously remained stationary started rotating, while the other wheel stopped. Based on this observation, we suspected that the L293D motor driver IC might be faulty. Therefore, we replaced the L293D and uploaded the code again to test the circuit.

Initially, the motors did not rotate when the starting speed limit was set to 150 in the code. AI said that the limit was too low and the motor needed a higher value to start. We increased the speed limit to 180 and uploaded the modified code. Thankfully, the circuit started working, and the UGV began moving successfully.

We used the following code to test the functionality without bluetooth:

```
// ---------------- Pins (Matches your diagram configuration) ----------------
const int IN1 = 32, IN2 = 33, ENA = 27;   // Left motor  (j9, j4, j10)
const int IN3 = 25, IN4 = 26, ENB = 13;   // Right motor (a4, a9, a3)

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);

  // Attach standard high PWM values to Enable lines to keep them fully on
  #if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
    ledcAttach(ENA, 1000, 8); ledcAttach(ENB, 1000, 8);
    ledcWrite(ENA, 200); ledcWrite(ENB, 200);
  #else
    ledcSetup(0, 1000, 8); ledcSetup(1, 1000, 8);
    ledcAttachPin(ENA, 0); ledcAttachPin(ENB, 1);
    ledcWrite(0, 200); ledcWrite(1, 200);
  #endif
}

void loop() {
  // 1. Move BOTH Forward
  digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  delay(2000);

  // 2. STOP
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  delay(2000);

  // 3. Move BOTH Backward
  digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
  delay(2000);

  // 4. STOP
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
  delay(2000);
}
```

After seeing that this code was working, we uploaded the final code for operating via bluetooth using the dabble app.

```
#define CUSTOM_SETTINGS
#define INCLUDE_GAMEPAD_MODULE

#include <DabbleESP32.h>

#if __has_include(<esp_arduino_version.h>)
#include <esp_arduino_version.h>
#endif

// ---------------- MOTOR PINS ----------------

// Left motor
#define IN1 32
#define IN2 33
#define ENA 27

// Right motor
#define IN3 25
#define IN4 26
#define ENB 13

// ---------------- PWM SETTINGS ----------------

#define PWM_FREQ 1000
#define PWM_RES  8

#define MIN_DUTY 100
#define MAX_DUTY 255

int speedDuty = 180;
const int STEP = 35;

// ---------------- PWM COMPATIBILITY ----------------

void pwmWriteMotor(int pin, int channel, int duty) {
  duty = constrain(duty, 0, 255);

#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(pin, duty);
#else
  ledcWrite(channel, duty);
#endif
}

void pwmSetupMotor(int pin, int channel) {
#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(pin, PWM_FREQ, PWM_RES);
#else
  ledcSetup(channel, PWM_FREQ, PWM_RES);
  ledcAttachPin(pin, channel);
#endif
}

// ---------------- MOTOR CONTROL ----------------

void setMotor(int in1, int in2, int en,
              int channel, float speed) {
```

```
  speed = constrain(speed, -1.0, 1.0);

  if (speed == 0) {
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    pwmWriteMotor(en, channel, 0);
    return;
  }

  if (speed > 0) {
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
  } else {
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
  }

  int duty = MIN_DUTY +
             abs(speed) * (speedDuty - MIN_DUTY);

  duty = constrain(duty, MIN_DUTY, speedDuty);

  pwmWriteMotor(en, channel, duty);
}

void drive(float leftSpeed, float rightSpeed) {
  setMotor(IN1, IN2, ENA, 0, leftSpeed);
  setMotor(IN3, IN4, ENB, 1, rightSpeed);
}

void stopMotors() {
  drive(0, 0);
}

// ---------------- SPEED CONTROL ----------------

void updateSpeed() {

  if (GamePad.isTrianglePressed()) {
    speedDuty += STEP;
    speedDuty = min(speedDuty, MAX_DUTY);
    delay(180);
  }

  if (GamePad.isCrossPressed()) {
    speedDuty -= STEP;
    speedDuty = max(speedDuty, MIN_DUTY);
    delay(180);
  }
}

// ---------------- SETUP ----------------

void setup() {

  Serial.begin(115200);

  pinMode(IN1, OUTPUT);
```

```
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  pwmSetupMotor(ENA, 0);
  pwmSetupMotor(ENB, 1);

  stopMotors();

  Dabble.begin("UGV_ESP32");

  Serial.println("UGV Ready!");
}

// ---------------- MAIN LOOP ----------------

void loop() {

  Dabble.processInput();

  updateSpeed();

  float leftSpeed = 0;
  float rightSpeed = 0;

  // -------- JOYSTICK MODE --------

  int x = GamePad.getXaxisData();
  int y = GamePad.getYaxisData();

  if (GamePad.getRadius() > 0) {

    float forward = y / 7.0;
    float turn = x / 7.0;

    const float TURN_GAIN = 0.8;

    leftSpeed  = forward + turn * TURN_GAIN;
    rightSpeed = forward - turn * TURN_GAIN;

    leftSpeed = constrain(leftSpeed, -1.0, 1.0);
    rightSpeed = constrain(rightSpeed, -1.0, 1.0);

  }

  // -------- D-PAD MODE --------

  else {

    if (GamePad.isUpPressed()) {
      leftSpeed = 1;
      rightSpeed = 1;
    }

    else if (GamePad.isDownPressed()) {
      leftSpeed = -1;
      rightSpeed = -1;
    }
```

```
    else if (GamePad.isLeftPressed()) {
      leftSpeed = -1;
      rightSpeed = 1;
    }

    else if (GamePad.isRightPressed()) {
      leftSpeed = 1;
      rightSpeed = -1;
    }

    // -------- SPIN BUTTONS --------

    if (GamePad.isSquarePressed()) {
      leftSpeed = -1;
      rightSpeed = 1;
    }

    if (GamePad.isCirclePressed()) {
      leftSpeed = 1;
      rightSpeed = -1;
    }
  }

  // -------- DRIVE --------

  drive(leftSpeed, rightSpeed);

  delay(20);
}
```

https://youtu.be/ZePPmzlz79Q?si=0x5sTOkgIl5HCet6

this is the link of our working circuit